Day 1

Monday, 22 June 2026 9:23 AM

Python

.c → .out / .exe → 1100111 / 1100100

.py → .pyc → pvm → 0110100

int n = 10;

$\boxed{n = 10}$

int	boolean
char	short
float	long
double	byte

int
float
boolean
string
→ DT: Python

int → 4 bytes → 32 bits → 31 bits

-2^{31} to $2^{31}-1$

int
float
string
boolean
→ DT in Python

Operators

Arithmetic → + - * / % // **

Assignment → += , -=, ...

Bitwise → & | ^

Comparison

Logical → and or not

> >= < <=
== !=

$2^3\ 2^2\ 2^1\ 2^0$

0 0 1 0 &

0 0 0

→ (^) XOR

0 1 0 1
0 0 0 0

01	1
10	1
11	0

0 0 1 0 → 2
^ 0 1 0 1 → 5
0 1 1 1 → 7

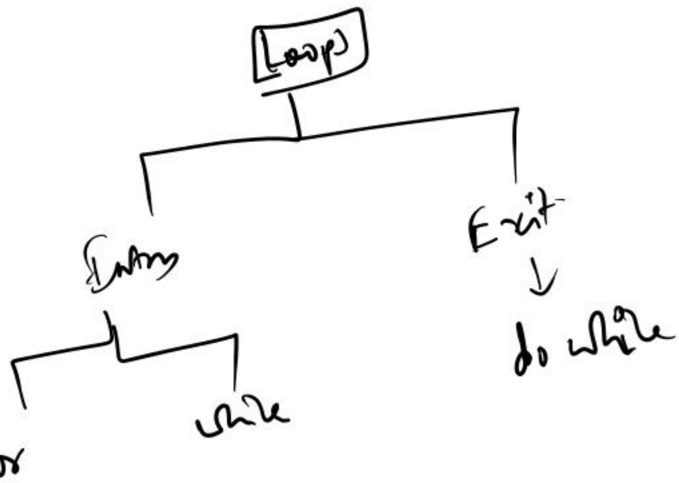
n $\boxed{81}$ → Divisible by 3

n $\boxed{120}$ → Divisible by 3 & 5

n $\boxed{40}$ → Divisible by 5

n $\boxed{7}$ → Not Divisible.

Get n as input
check for divisibility
? 3 & 5




```
for(int i=0; i<n; i++){
```

}

for i in range(n):

for i in range(1, n):

end → exclusive

start → inclusive

for i in range(1, n, 2):

n = 4

1 × 4 = 4
2 × 4 = 8
3 × 4 = 12
⋮
10 × 4 = 40

n = 9

1 × 9 = 9
2 × 9 = 18
3 × 9 = 27
⋮
10 × 9 = 90

n = 4

_ _ _ *
_ _ * *
_ * * *
* * * *

n = 5

_ _ _ _ *
_ _ _ * *
_ _ * * *
_ * * * *
* * * * *

1 No. of rows → n

→ n → space + stars

② No of cols in ith row -

③ What needs to be printed? → space + stars.

n = 4

```
① → _ _ _ *
② → _ _ * *
③ → _ * * *
④ → * * * *
```

i	spaces	stars
1	3	1
2	2	2
3	1	3
4	0	4

spaces → $n - i$

stars → i

n = 4

```
_ _ _ *
_ _ * * *
_ * * * * *
* * * * * * *
_ * * * * *
_ _ * * *
_ _ _ *
```

n = 3

```
_ _ *
_ * * *
* * * * *
_ * * *
_ _ *
```

n = 4

```
① _ _ _ *
② _ _ * * *
③ _ * * * * *
④ * * * * * * *
③ _ * * * * *
② _ _ * * *
① _ _ _ *
```

First half

second half

i	spaces	stars
1	3	1
2	2	3
3	1	5
4	0	7

spaces → $n - i$

→ $(2i - 1)$

stars ->

```python
n = int(input("Enter n : "))
# Diamond pattern
# First half
for i in range(1, n + 1) :
    # Spaces -> n - i
    for j in range(1, n - i + 1):
        print("_", end = " ")
    # Stars -> 2 * i - 1
    for j in range(1, 2 * i) :
        print("*", end = " ")
    print()
```

```python
# Second half
for i in range(n - 1, 0, -1) :
    # Spaces -> n - i
    for j in range(1, n - i + 1):
        print("_", end = " ")
    # Stars -> 2 * i - 1
    for j in range(1, 2 * i) :
        print("*", end = " ")
    print()
```

n = 4

1 3 5 7
3 5 7 1
5 7 1 3
7 1 3 5

n = 5

1 3 5 7 9
3 5 7 9 1
5 7 9 1 3
7 9 1 3 5
9 1 3 5 7

n = 4

1 3 5 7 -> 0
3 5 7 1 -> 1
5 7 1 3 -> 2
7 1 3 5 -> 3

num -> 2i+1

end -> 2n-1

```python
n = int(input("Enter n : "))
```

```python
# Number pattern
end = (2 * n) - 1
for i in range(n) :
    num = (2 * i) + 1
    for j in range(n) :
        print(num, end = " ")
        num += 2
        if num > end :
            num = 1
    print()
```

n = 3

3 3 3 3 3
3 2 2 2 3
3 2 1 2 3
3 2 2 2 3
3 3 3 3 3

n = 2

2 2 2
2 1 2
2 2 2

n = 4

Col

Rows

Rows / Col	0	1	2	3	4	5	6
0	4	4	4	4	4	4	4
1	4	3	3	3	3	3	4
2	4	3	2	2	2	3	4
3	4	3	2	1	2	3	4
4	4	3	2	2	2	3	4
5	4	3	3	3	3	3	4
6	4	4	4	4	4	4	4

n [5]

	0	1	2	3	4	5	6	7	8
0	5	5	5	5	5	5	5	5	5
1	5	4	4	4	4	4	4	4	5
2	5	4	3	3	3	3	3	4	5
3	5	4	3	2	2	2	3	4	5
4	5	4	3	2	1	2	3	4	5
5	5	4	3	2	2	2	3	4	5
6	5	4	3	3	3	3	3	4	5
7	5	4	4	4	4	4	4	4	5
8	5	5	5	5	5	5	5	5	5

n [4]

2n-1

k [7]

	0	1	2	3	4	5	6
0	4	$\boxed{4}$	4	4	4	4	4
1	4	3	3	3	3	3	4
2	4	3	2	2	2	3	4
3	4	3	2	1	2	3	4
4	4	3	2	2	2	3	4
5	4	3	3	3	3	3	4
6	4	4	4	4	4	4	4

n (4)

$4 \to 2(4) - 1$

$3 \to 2(3) - 1$

$2 \to 2(2) - 1$

$1 \to 2(1) - 1$

	Row	col
4 →	1, k	1, k
3 →	2, k-1	2, k-1

i [0] j [1]

top [0] left [1] right [5]

down [6]

top → i Right → k − 1 − j

left → j Down → k − 1 − i

7 − 1 − 1

7 − 1 − 0

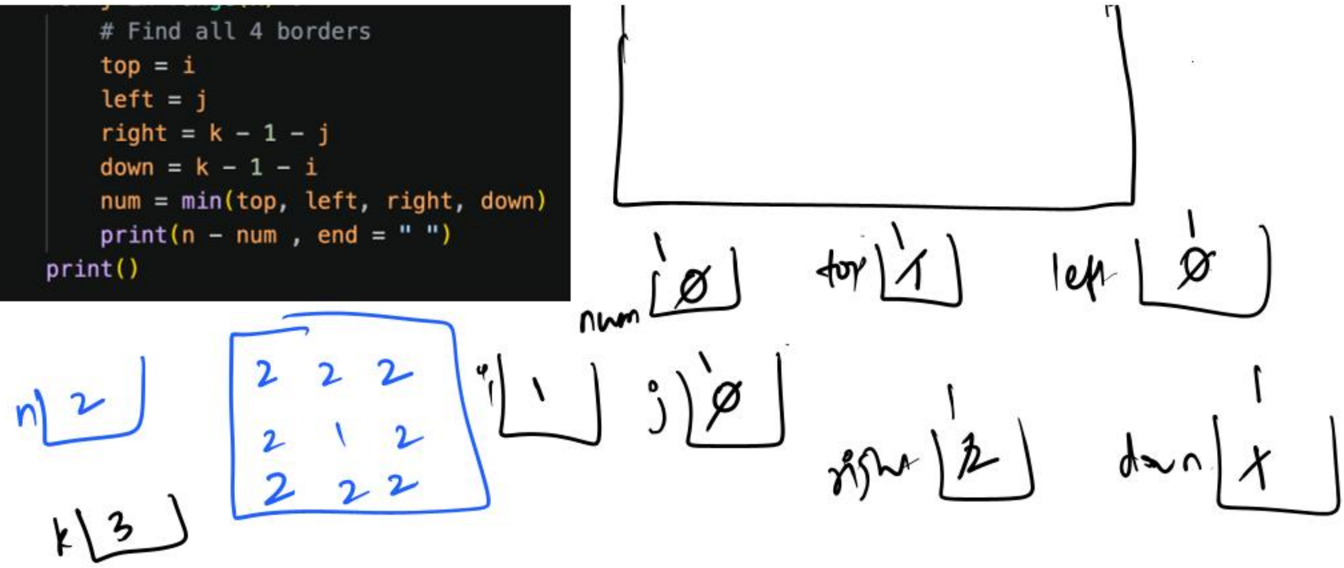
```
n = int(input("Enter n : "))
# Number pattern
k = (2 * n) - 1
for i in range(k) :
    for j in range(k) :
```

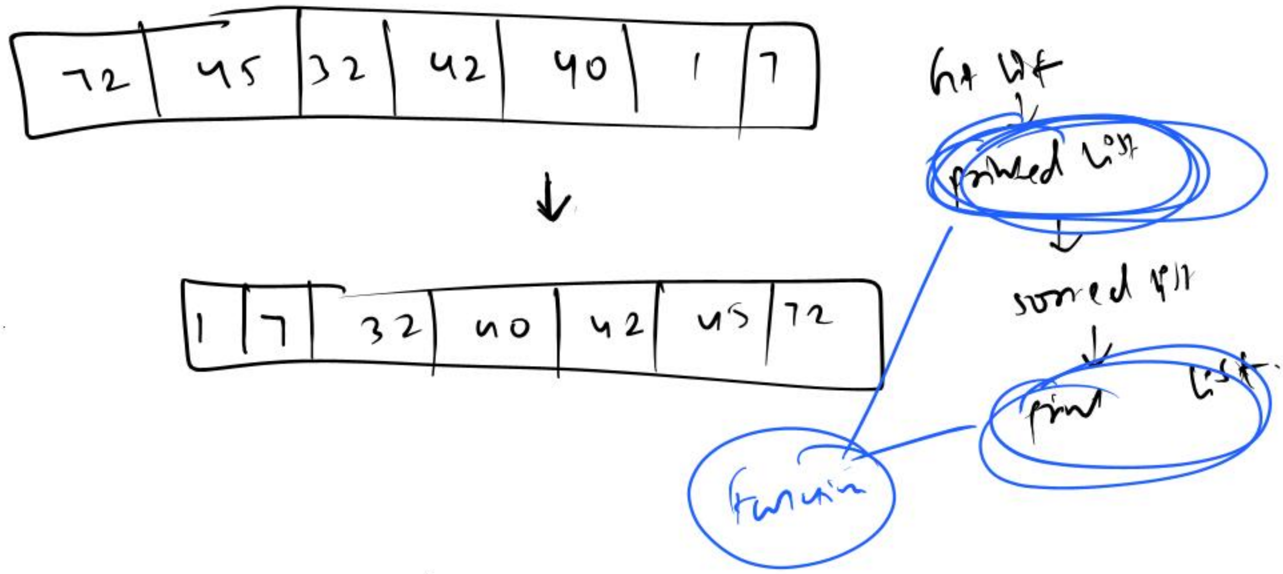
2 2 2

2 1

→ Continue

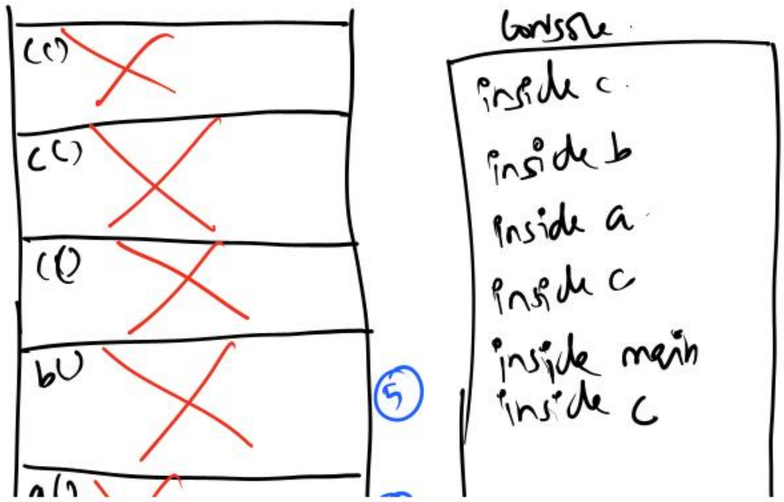
```python
        # Find all 4 borders
        top = i
        left = j
        right = k - 1 - j
        down = k - 1 - i
        num = min(top, left, right, down)
        print(n - num , end = " ")
    print()
```





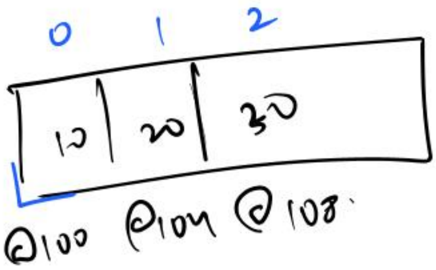
Functions → wrap code → Reuse the code.

```python
def c() :
    print("inside c")

def b() :
    c()
    print("inside b")

def a() :
    b()
    print("inside a")
```



```
11        c()
12
13    a()
14    print("inside main")
15    c()
```

List → collection of elements → index → $\boxed{0 \text{ to } n-1}$



arr @100

$arr[2] = BA + (2 * 4)$

$$\boxed{arr[i] = BA + (i * size)}$$

"10 20 30" → ["10", "20", "30"]